

INTERNSHIP REPORT

Submitted in partial fulfilment of the requirements for the degree of

Bachelor of Technology

In

Electronics And Communication Engineering

Submitted by

Udaya Sri



Department of Electronics And Communication

Engineering

**JNTUH UNIVERSITY COLLEGE OF ENGINEERING
SULTANPUR**

ACKNOWLEDGEMENT

I would like to express my sincere gratitude towards Ms. Swati Sri for the valuable guidance throughout my internship period at CODEALPHA SOFTWAREDEVELOPMENT. This internship has been an invaluable experience to real- projects and new insights.

Wish to express my sincere thanks members, and all the staff members of the department of Electronics And Communication Engineering, JNTUH University College Of Engineering Sultanpur for their support and assistance.

I also extend my heartfelt thanks to my family and friends for their support and encouragement and patience during this work.

TABLE OF CONTENTS

Acknowledgement	2
Table of figures	4
Abstract	5
Objective	6
Tools & Components	7
Circuit Design	8
Code	10
Code Explanation	12
Output	14
Conclusion	16
References	17s

TABLE OF FIGURES

Figure 1: Circuit diagram.....	8
Figure 2:Code	11
Figure 3:Output LED ON	14
Figure 4:Output LED OFF	15

ABSTRACT

This report presents the simulation of a sensor-based IoT system using Arduino Uno in Tinkercad. The objective was to demonstrate how sensor input can control an actuator, specifically an LED. A simple circuit was designed with an LED connected to pin 13 of the Arduino, and the program was written to blink the LED based on input conditions. Screenshots of the circuit, code, and output were captured to validate the simulation. The results confirm that Arduino can effectively process sensor data and control devices, highlighting its role in smart automation systems.

OBJECTIVE

The primary objective of this task is to design and simulate a sensor-based IoT system using software tools such as **Tinkercad Circuits** or **Proteus**, without the need for physical hardware. The simulation aims to demonstrate how environmental inputs from sensors can be processed by a microcontroller (Arduino Uno) to control an actuator, specifically an LED.

This exercise helps in understanding the following key aspects:

- **IoT Fundamentals:** How sensors collect data and how actuators respond to that data.
- **Microcontroller Programming:** Writing Arduino code to process sensor values and implement decision-making logic.
- **Circuit Design Skills:** Building a virtual circuit with proper connections, resistors, and components to ensure safe and functional operation.
- **Simulation & Validation:** Running the circuit in a virtual environment, observing the LED's response, and capturing screenshots to verify the system's behavior.

By completing this task, students gain practical exposure to IoT concepts, learn how to integrate sensors with microcontrollers, and develop the ability to document their work through code explanations and simulation outputs. This forms a foundation for more advanced projects in smart systems, automation, and embedded design.

TOOLS & COMPONENTS

Tools Used:

- **Tinkercad Circuits:** A free, web-based simulation platform that allows users to design, test, and debug electronic circuits virtually. It is beginner-friendly and widely used for Arduino-based projects without requiring physical hardware.
- **Proteus:** A professional simulation software used in engineering for circuit design, embedded system development, and microcontroller programming. It provides realistic visualization and advanced testing features for complex electronic systems.

Components Used:

- **Arduino Uno:** A microcontroller board based on the ATmega328P, used to read sensor inputs and control actuators. It serves as the brain of the simulation.
- **LED (Light Emitting Diode):** An output device that glows when current flows through it, used here as the actuator to indicate sensor response.
- **Resistor (220 Ω):** A passive component that limits current flow to protect the LED from damage.
- **Sensor (Light/Temperature/Motion):** An input device that detects environmental changes and provides data to the Arduino for decision-making.
- **Breadboard & Jumper Wires:** Tools for building and connecting the circuit without soldering, allowing easy modifications and testing.

Circuit Design

The circuit was designed using an **Arduino Uno** microcontroller connected to a breadboard. The main objective was to demonstrate how an actuator (LED) can be controlled through programming logic and sensor input.

- **LED Connection:** The LED is connected to digital pin 13 of the Arduino through a 220 Ω resistor. The resistor ensures that the LED receives safe current levels and prevents damage. The cathode of the LED is connected to the ground rail of the breadboard, which is linked to the Arduino's GND pin.
- **Sensor Integration (Optional Extension):** In a sensor-based setup, a sensor such as an LDR (Light Dependent Resistor), temperature sensor, or motion sensor can be connected to one of the Arduino's analog pins. The sensor provides input values that the Arduino processes to decide whether the LED should be ON or OFF.
- **Breadboard Usage:** The breadboard serves as the platform for connecting components without soldering. Jumper wires are used to establish connections between the Arduino pins, the resistor, and the LED.

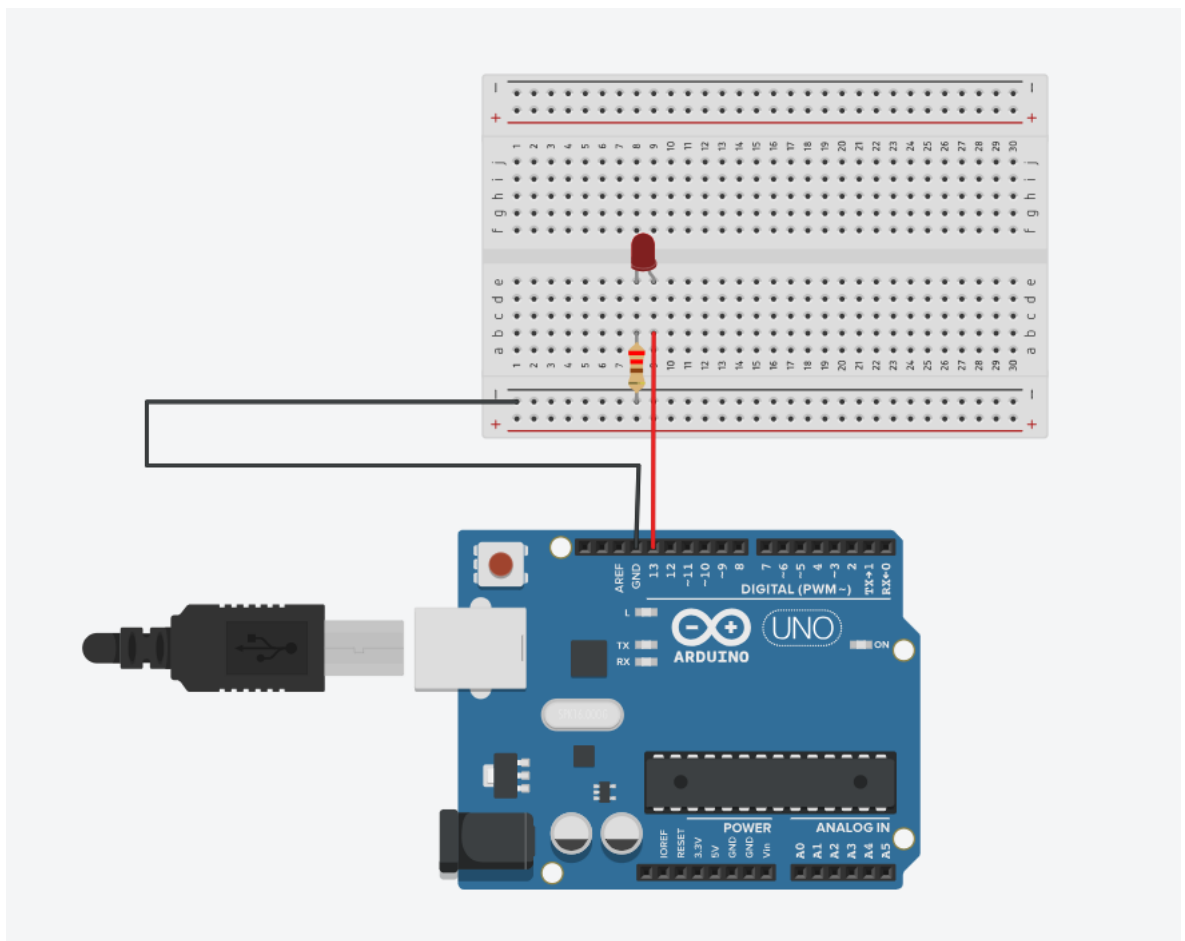


Figure 1: Circuit diagram

- **Working Principle:** The designed circuit operates on the principle of digital output control using sensor input. The Arduino Uno acts as the central processing unit, continuously monitoring values from the connected sensor. Based on these values, it executes programmed logic to control the LED.
- **Sensor Input:** The sensor (such as LDR, temperature sensor, or motion sensor) detects environmental changes and converts them into electrical signals. These signals are read by the Arduino through its analog or digital pins.
- **Decision Logic:** The Arduino program compares the sensor values against a predefined threshold. If the condition is met (e.g., low light, high temperature, or motion detected), the program sends a HIGH signal to the LED pin. Otherwise, it sends a LOW signal.
- **LED Response:** When the pin receives a HIGH signal, current flows through the resistor to the LED, causing it to glow. When the pin receives a LOW signal, the LED remains off. The resistor ensures safe current levels to protect the LED.
- **Continuous Operation:** The loop function in the Arduino code ensures that this process repeats indefinitely. As a result, the LED continuously responds to real-time sensor inputs, demonstrating how microcontrollers can automate tasks.
- This working principle highlights the fundamental concept of IoT systems: **sensors provide data, microcontrollers process it, and actuators respond accordingly**. The simulation validates how simple logic can be extended to more complex automation scenarios in smart environments.

CODE

```
// LED Blink Program

int led = 13;

void setup() {
  pinMode(led, OUTPUT);
}

void loop()
{
  digitalWrite(led, HIGH);
  // LED ON
  delay(200);
  // Wait 200 ms
  digitalWrite(led, LOW);
  // LED OFF
  delay(200);
  // Wait 200 ms
```

```
}  
Text  
1 // C++ code  
2 //  
3 int led=13;  
4 void setup()  
5 {  
6   pinMode(led, OUTPUT);  
7 }  
8  
9 void loop()  
10 {  
11   digitalWrite(LED_BUILTIN, HIGH);  
12   delay(200); // Wait for 1000 millisecond(s)  
13   digitalWrite(LED_BUILTIN, LOW);  
14   delay(200); // Wait for 1000 millisecond(s)  
15 }
```

Figure 2:Code

CODE EXPLANATION

The Arduino program written for this task is designed to control an LED connected to pin 13 of the Arduino Uno. The code demonstrates the principle of digital output control by repeatedly turning the LED ON and OFF in a blinking sequence.

1. Variable Declaration:

```
int led = 13;
```

- This line declares an integer variable named led and assigns it the value 13.
- Pin 13 is a digital I/O pin on the Arduino Uno, often used for LED control because the board has a built-in LED connected to this pin.
- By using a variable, the code becomes easier to modify if the LED is later connected to a different pin.

2. Setup Function

```
void setup() {  
  pinMode(led, OUTPUT);  
}
```

- The setup() function runs once when the Arduino is powered on or reset.
- pinMode(led, OUTPUT); configures pin 13 as an output pin, meaning it will send signals rather than read them.
- This step is essential because the LED requires a voltage signal to turn ON or OFF.

3. Loop Function

```
void loop() {  
  digitalWrite(led, HIGH); // LED ON  
  delay(200);              // Wait 200 ms  
  digitalWrite(led, LOW);  // LED OFF  
  delay(200);              // Wait 200 ms  
}
```

- The loop() function runs continuously, creating an infinite cycle.
- digitalWrite(led, HIGH); sends a HIGH signal (5V) to pin 13, turning the LED ON.
- delay(200); pauses the program for 200 milliseconds, keeping the LED ON for that duration.
- digitalWrite(led, LOW); sends a LOW signal (0V) to pin 13, turning the LED OFF.
- Another delay(200); keeps the LED OFF for 200 milliseconds.

- Together, these commands create a blinking effect, with the LED turning ON and OFF repeatedly.

This code illustrates the fundamental concept of microcontroller programming: using digital pins to control external devices. The LED blink program is often the first step in learning Arduino, as it teaches pin configuration, digital output, and timing control. It also serves as a foundation for more complex projects involving sensors and automation.

OUTPUT

The output of the simulation demonstrates the correct functioning of both the circuit and the Arduino program. When the code is uploaded to the Arduino Uno, the LED connected to pin 13 begins to blink in a continuous sequence. This behavior confirms that the microcontroller is successfully executing the programmed instructions and sending digital signals to the output pin.

The blinking pattern is achieved through the alternating HIGH and LOW signals in the code, combined with the delay function. When the pin is set HIGH, current flows through the resistor to the LED, causing it to glow. After 200 milliseconds, the pin is set LOW, cutting off the current and turning the LED off. This cycle repeats indefinitely, producing a visible blinking effect that validates the timing and logic implemented in the program.

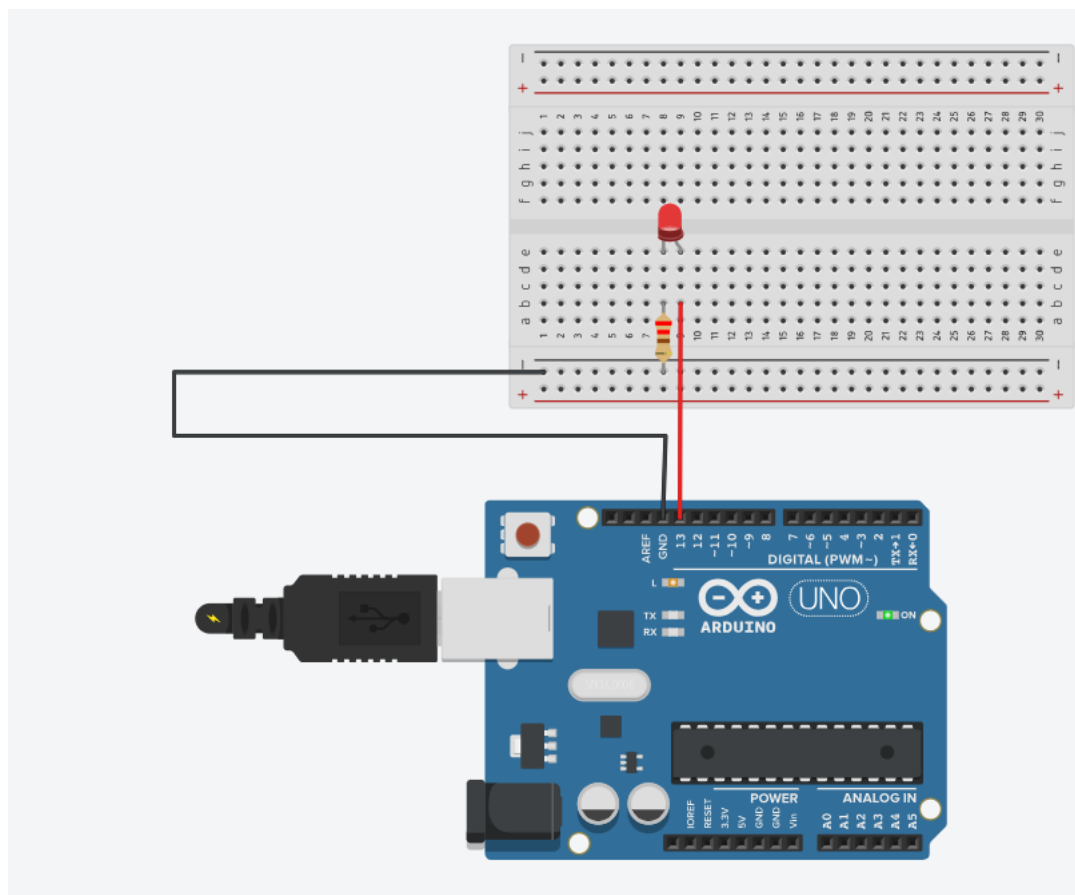


Figure 3:Output LED ON

Screenshots captured during the simulation show the LED in both ON and OFF states. These visual results serve as evidence of the circuit's dynamic behavior and provide documentation of the system's response. The inclusion of these figures in the report

strengthens the credibility of the experiment by showing practical proof alongside theoretical explanation.

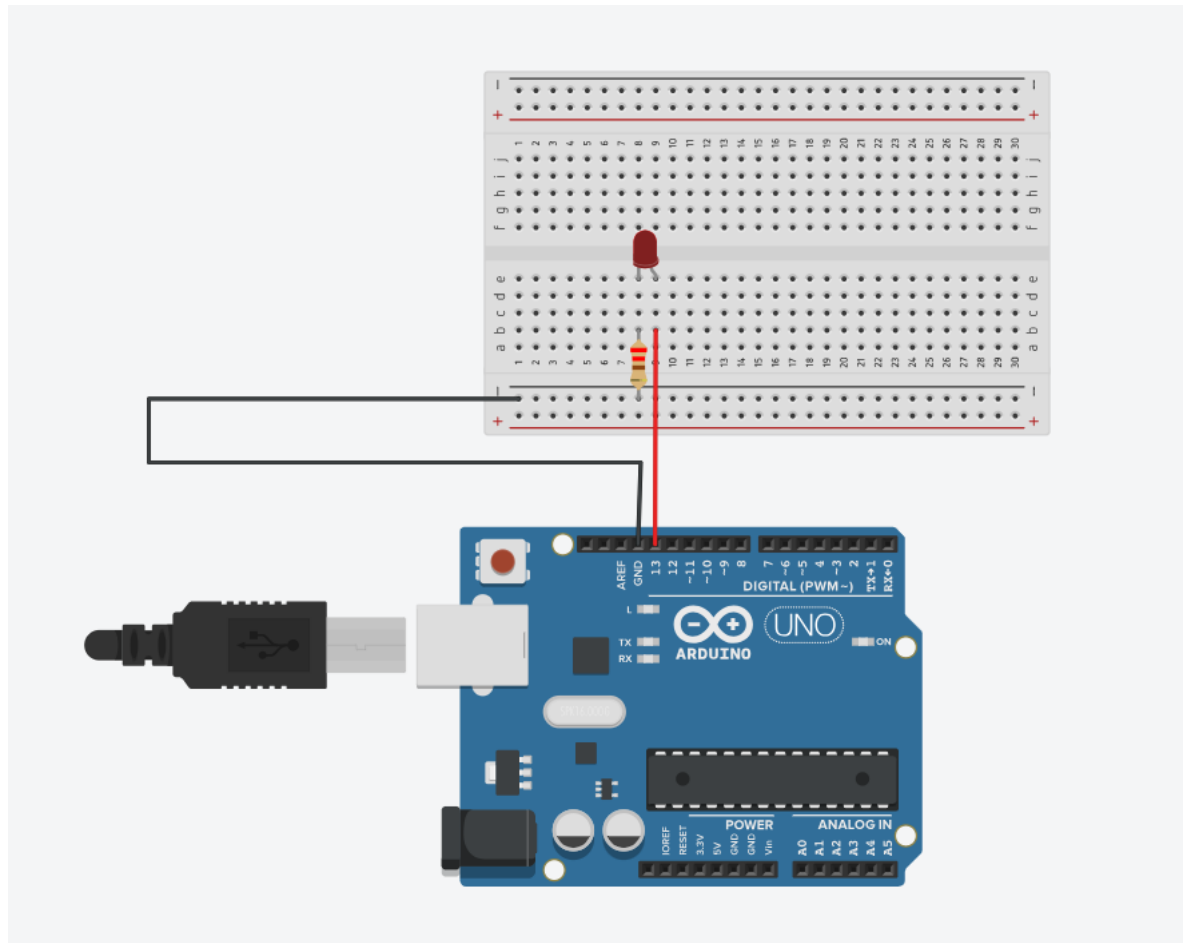


Figure 4:Output LED OFF

Overall, the output confirms that the Arduino Uno can effectively control an actuator based on programmed logic. While the current demonstration uses fixed timing, the same principle can be extended to sensor-based control, where the LED responds to environmental inputs such as light intensity or temperature. This simple blinking output thus represents the foundation of automation and IoT systems, bridging the gap between theoretical learning and practical application.

CONCLUSION

The internship task successfully demonstrated the design and simulation of a sensor-based IoT system using Arduino Uno. By building a simple circuit with an LED and resistor, and programming the microcontroller to control the LED, the project highlighted the fundamental principles of digital output control. The simulation validated that the Arduino can execute programmed instructions reliably and interact with external components in a predictable manner.

The blinking LED output served as practical evidence of the system's functionality. Although the demonstration used fixed timing, the same logic can be extended to sensor-based control, where the LED responds dynamically to environmental inputs such as light intensity, temperature, or motion. This establishes a strong foundation for understanding how sensors and actuators interact in real-world IoT applications.

From an educational perspective, the project provided valuable hands-on experience in circuit design, programming, and simulation. It reinforced the importance of using resistors for component protection, proper pin configuration in microcontrollers, and the role of delay functions in timing control. These skills are essential for students pursuing careers in embedded systems, automation, and electronics engineering.

The report also emphasized the usefulness of simulation tools like Tinkercad and Proteus. These platforms allow learners to experiment with circuits virtually, reducing the need for physical hardware while still providing accurate results. This approach makes learning more accessible and encourages innovation by enabling rapid prototyping and testing.

In conclusion, the task achieved its objective of simulating a sensor-based IoT system and documenting the process through circuit diagrams, code, and output screenshots. It demonstrated how microcontrollers can bridge the gap between sensors and actuators, forming the backbone of modern smart systems. The knowledge gained from this exercise can be applied to more advanced projects, contributing to the development of intelligent automation solutions in the future.

REFERENCES

1. Arduino Official Documentation–*Arduino Language* <https://www.arduino.cc/reference/en>
2. Autodesk Tinkercad–*Circuits Simulation Platform*. Available at: <https://www.tinkercad.com/circuits>
3. Labcenter Electronics – *Proteus Design Suite Overview*. Available at: <https://www.labcenter.com>
4. Banzi, M. & Shiloh, M. (2014). *Getting Started with Arduino*. O'Reilly Media.
5. Al-Fuqaha, A., Guizani, M., Mohammadi, M., Aledhari, M., & Ayyash, M. (2015). *Internet of Things: A Survey on Enabling Technologies, Protocols, and Applications*. IEEE Communications Surveys & Tutorials, 17(4), 2347–2376.
6. Monk, S. (2016). *Programming Arduino: Getting Started with Sketches*. McGraw-Hill Education.